

5 SQL Window Functions

Every Data Engineer Should Know

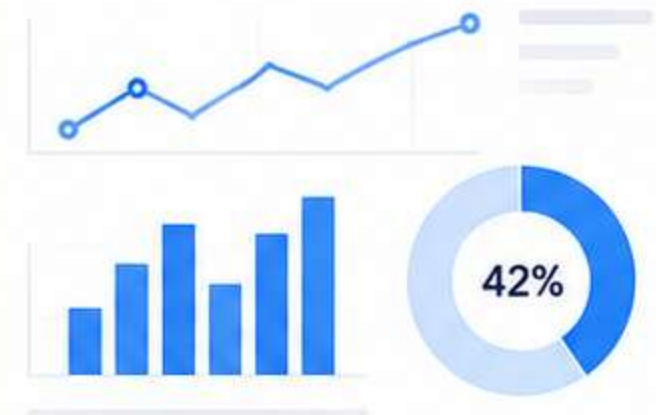
Essential SQL concepts used in real-world ETL pipelines, analytics dashboards, and Databricks workflows

user_id	order_date	amount
101	2024-01-01	120.00
101	2024-01-05	80.00
101	2024-01-10	150.00
102	2024-01-03	200.00
102	2024-01-07	50.00
...	...	...



user_id	order_date	amount	running_total
101	2024-01-01	120.00	120.00
101	2024-01-05	80.00	200.00
101	2024-01-10	150.00	350.00
102	2024-01-03	200.00	200.00
102	2024-01-07	50.00	250.00
...	...	...	...

Analytics Dashboard



The language of data



Distributed processing at scale



databricks

Unified lakehouse for data + AI



Master window functions.

Write smarter queries. Build better pipelines. Deliver deeper insights.



1. ROW_NUMBER()



Remove duplicates + select latest records

SQL

```
SELECT *,  
    ROW_NUMBER() OVER(  
        PARTITION BY customer_id  
        ORDER BY order_date DESC  
    ) as rn  
FROM orders;
```

Raw Data (with Duplicates)

order_id	customer_id	order_date	amount
101	C001	2024-05-01	120.00
102	C001	2024-05-03	150.00
103	C001	2024-05-03	150.00
104	C002	2024-05-02	200.00
105	C002	2024-04-28	180.00
106	C003	2024-05-01	90.00

ROW_NUMBER()

PARTITION BY customer_id
ORDER BY order_date DESC



Latest Record per Customer

order_id	customer_id	order_date	amount
103	C001	2024-05-03	150.00
104	C002	2024-05-02	200.00
106	C003	2024-05-01	90.00



Keeps the most recent record
for each customer



Use **ROW_NUMBER()** to rank rows within a partition
and filter the latest (or first) record.

2. RANK() / DENSE_RANK()



Used for ranking users, products, sales

SQL

```
SELECT employee_name,
       sales,
       RANK() OVER(
         ORDER BY sales DESC
       ) as rank
FROM sales_data;
```

Raw Sales Data

employee_name	sales
Alice	9500
Bob	7500
Charlie	7500
David	5000
Eve	3000



Sales Leaderboard (Using RANK())

Rank	employee_name	sales
 1	Alice	9500
 2	Bob	7500
 2	Charlie	7500
4	David	5000
5	Eve	3000



RANK() leaves gaps in ranking when there are ties. Use **DENSE_RANK()** if you want consecutive ranks without gaps.



Great for leaderboards, top N analysis, performance tracking, and reporting.



3. LAG()



Compare current row with previous row

SQL

```
SELECT month,
       revenue,
       LAG(revenue) OVER(
         ORDER BY month
       ) as previous_revenue
FROM monthly_sales;
```

→ LAG() fetches the value from the previous row in the ordered set.

→ Helps in trend analysis, period-over-period comparison, and change tracking.

Query Result

month	revenue	previous_revenue
Jan	10000	NULL
Feb	12000	10000
Mar	15000	12000
Apr	13000	15000
May	17000	13000
Jun	19000	17000

Month-over-Month Revenue Comparison



LAG() is perfect for identifying changes, growth trends, and deviations in time-series data.



4. LEAD()



Look ahead to next row values

SQL

```
SELECT event_id,  
       event_date,  
       LEAD(event_date) OVER(  
         ORDER BY event_date  
       ) as next_event  
FROM events;
```



LEAD() returns the value from the following row in the ordered result set.



Useful for forecasting, sessionization, gaps & intervals, and journey analysis.

Query Result

event_id	event_date	next_event
1	2024-05-01	2024-05-05
2	2024-05-05	2024-05-10
3	2024-05-10	2024-05-15
4	2024-05-15	2024-05-20
5	2024-05-20	NULL

Event Timeline (Look Ahead)



LEAD() helps you see what's coming next — perfect for forward-looking analysis.



5. SUM() OVER(PARTITION BY)



Running totals & cumulative metrics

SQL

```
SELECT order_date,
       revenue,
       SUM(revenue) OVER(
         ORDER BY order_date
       ) as running_total
FROM orders;
```

Query Result

order_date	revenue	running_total
2024-01-01	10,000	10,000
2024-01-02	15,000	25,000
2024-01-03	12,000	37,000
2024-01-04	20,000	57,000
2024-01-05	18,000	75,000
2024-01-06	25,000	100,000



Calculates a **running total** of revenue over time.



Perfect for cumulative analysis, growth tracking, and financial reporting.

Cumulative Revenue Growth



Total Revenue
100,000
↗ +25.6%
vs previous period



Avg. Daily Revenue
16,667
↗ +12.4%
vs previous period



Growth Trend
Strong
Upward



Max Daily Revenue
25,000
on 2024-01-06



SUM() OVER (ORDER BY ...) gives you a running total to track performance and growth over time.

